

Lab 4: 基于UDP的可靠传输

实验简介

本实验实现了一个基于 UDP 的简单“文件传输（FTP-like）”系统，并在应用层实现了两类可靠性重传策略——Go-Back-N (GBN) 与 Selective Repeat (SR)，以及两类拥塞控制算法——基于丢包的 Reno 与基于延迟的 Vegas。系统支持客户端/服务端握手、分片传输、滑动窗口、ACK/超时与快速重传、以及传输完成后的 MD5 完整性校验。为便于实验分析，还在客户端与服务端两侧记录了传输指标（吞吐量与流量利用率）。

目标：

- 在不可靠的 UDP 之上，分别实现 GBN 与 SR 的可靠传输。
- 在传输过程中接入 Reno 与 Vegas 拥塞控制，动态调整发送窗口（`cwnd`）。
- 对比不同组合（GBN/SR × Reno/Vegas）下的传输行为与性能。

运行环境

模拟环境：

- Windows 主机运行服务端，默认在 `0.0.0.0:9000` 开启服务，文件存储地址为 `/stor`，指令：`py -3 udpftp_server.py`
- WSL 运行客户端，文件在 `udpftp_client.py` 所在文件夹，指令：`python3 udpftp_client.py --server-host <server-host> --server-port <server-port> --op <download/upload> --file ./<client-file-name> --remote-name <server-file-name> --algo <sr/gbn> --cc <reno/vegas>`

真实环境：

- MacOS 运行客户端，指令同上
- 远程 Ubuntu 服务器运行服务端，指令 `python3 udpftp_server.py`

代码解释

项目结构（关键文件）：

- `udpftp/protocol.py`：协议与数据包定义（包头结构与辅助构造函数）。
- `udpftp/reliability.py`：可靠传输层，包含 GBN/SR 的发送与接收实现。
- `udpftp/congestion.py`：拥塞控制层，包含抽象基类与 Reno/Vegas 两种实现。
- `udpftp/utils.py`：工具函数（MD5、日志、时间、目录）。
- `udpftp_server.py`：服务端入口，处理上传/下载请求、会话线程与服务端侧统计。
- `udpftp_client.py`：客户端入口，处理命令行、握手与客户端侧统计。

数据包与协议（`udpftp/protocol.py`）：

- 包头字段：`seq`（序号）、`ack`（确认号）、`flags`（标志位）、`window`（接收窗口通告）、`ts_us`（时间戳微秒）、`length`（负载长度）。
- 标志位：`DATA`、`ACK`、`CTRL` 等；辅助函数：`make_data()`、`make_ack()`、`make_ctrl()`。
- MSS（默认 1200 字节）用于数据分片，包头与负载拼接成最终报文。

客户端/服务端握手与流程：

- 上传：客户端发送 `CMD UPLOAD <name> <size> <algo> <cc> <mss> <window>`；服务端回复 `OK` 后接收数据并存储，再返回服务器侧 MD5；客户端比对本地图件 MD5 验证。
- 下载：客户端发送 `CMD DOWNLOAD <name> 0 <algo> <cc> <mss> <window>`；服务端回复 `SIZE <n> MD5 <hex> OK`，随后发送数据；客户端完成后比对接收数据的 MD5 验证。

传输统计（增强）：

- 服务端：在上传/下载的 `Session.run()` 中包裹 `try-except-finally`，无论成功与否都记录统计并写入 `data.csv`。
- 客户端：通过 `CountingSocket` 统计发送/接收的总字节数，记录吞吐量与利用率到 `client_data.csv`。
- 指标定义：
 - 吞吐量 (B/s)：文件大小 / 传输时间。
 - 流量利用率：文件大小 / 发送或接收的总字节数（包含协议开销与重传）。

重传模块解释

文件：`udpftp/reliability.py`

整体结构：

- `Reliability(strategy="gbn"|"sr", mss=1200, timeout_ms=200, dup_ack_threshold=3)`：指定重传策略与分片大小等参数。
- 发送端接口：`send(sock, addr, data_bytes, cc, recv_packet, recv_adv_window)`。
- 接收端接口：`recv(sock, addr, total_size, send_adv_window)`。

发送端（核心逻辑）：

- 分片：根据 `mss` 将数据切成多个包，`total_packets = ceil(len(data)/mss)`。
- 窗口：`allowed_window() = min(cc.window(), recv_adv_window)`，其中 `cc.window()` 来自拥塞控制层，`recv_adv_window` 为对端通告的接收窗口大小。
- 发送循环：尽量填满允许窗口，将数据包放入 `send_buffer` 并记录发送时间。
- 等待 ACK：
 - 若提供 `recv_packet` 回调（服务端下载场景），用该回调拉取 ACK；否则在客户端发送端使用 `select.select([sock], ...)` 监听套接字，通过 `sock.recvfrom()` 取 ACK。
 - 每个 ACK 到达后调用 `cc.on_ack(rtt_ms, bytes_acked=mss)`，其中 RTT 通过发送时间差估算。
- 超时与重传：
 - GBN：超时重传“最早未确认”包；在重复 ACK 达到 `dup_ack_threshold` 时快速重传。
 - SR：超时重传“所有未确认的包”（按需），并独立维护 `acked` 状态。
- 结束：当 `base`（滑窗基序号）推进到 `total_packets`，传输完成，返回统计信息（总包数与持续时间）。

接收端（核心逻辑）：

- GBN：维护 `expected` 期望序号，收到期望包就递增并发送累计 ACK (`ack=expected`)，否则重发上一累计 ACK。接收端按序组装数据。

- SR: 允许乱序接收, 缓存到 `received[seq]`, 对每个到达的包单独发送 ACK (`ack=seq+1`), 并尝试从 `expected` 起连续刷新到有序输出。
- 完成: 当接收包数达到 `total_packets`, 返回完整数据 (裁剪至 `total_size`)。

与拥塞控制的互动:

- 发送端在每次 ACK 到达、超时时都会调用拥塞控制的 `on_ack()` 或 `on_timeout()`, 以更新 `cwnd` 并影响随后的发送窗口上限。

鲁棒性与兼容性:

- 发送端在客户端侧使用 `select.select([sock], ...)`, 因此自定义套接字包装器必须实现 `fileno()`; 代码已在客户端与服务端的包装器中补充该方法, 避免运行时错误。

拥塞控制模块解释

文件: `udpftp/congestion.py`

抽象基类 `CongestionControl`:

- 关键属性: `cwnd` (拥塞窗口, 按“包”为单位)、`ssthresh` (慢启动阈值)、`max_cwnd` (上限)。
- 核心方法:
 - `on_ack(rtt_ms, bytes_acked)`: ACK 到达时更新 `cwnd`。
 - `on_loss()` 与 `on_timeout()`: 发生丢包或超时调整 `cwnd` 与 `ssthresh`。
 - `window()`: 将 `cwnd` 截断到 `[1, max_cwnd]` 并返回整数窗口大小供发送端参考。

Reno 实现:

- 慢启动 (`cwnd < ssthresh`): 每个 ACK 将 `cwnd` 增加 1 (包)。
- 拥塞避免 (`cwnd ≥ ssthresh`): 每个 ACK 将 `cwnd` 增加约 $1/cwnd$ (包), 近似每 RTT 增加 1 包。
- 超时: `ssthresh = cwnd/2` (下限 2), `cwnd = 1`。
- 快速重传/恢复 (抽象为 `on_loss()`): `ssthresh = cwnd/2`, `cwnd = ssthresh`。

Vegas 实现:

- RTT 追踪: 维护 `base_rtt_ms` (历次最小 RTT)。每次 ACK 更新观测 RTT, 并刷新基准 RTT。
- 期望吞吐与实际吞吐: `expected = cwnd / base_rtt`, `actual = cwnd / rtt`, 差值 `diff = expected - actual`。
- 调整策略:
 - `diff < alpha`: 说明链路未充分利用, `cwnd += 1`。
 - `alpha ≤ diff ≤ beta`: 保持 `cwnd` 不变。
 - `diff > beta`: 说明链路拥塞倾向, `cwnd -= 1`。
- 丢包响应 (`on_loss()`): Vegas 相对温和, `cwnd = max(cwnd - 1, 1)`, 并将 `ssthresh = max(cwnd, 2)`。

与可靠层的耦合点:

- 发送端在每次处理 ACK 时调用 `cc.on_ack(rtt_ms, bytes_acked=mss)`, 在超时与重传时调用 `cc.on_timeout()`; 拥塞窗口通过 `cc.window()` 反馈给可靠层的允许发送上限。

参数与行为建议:

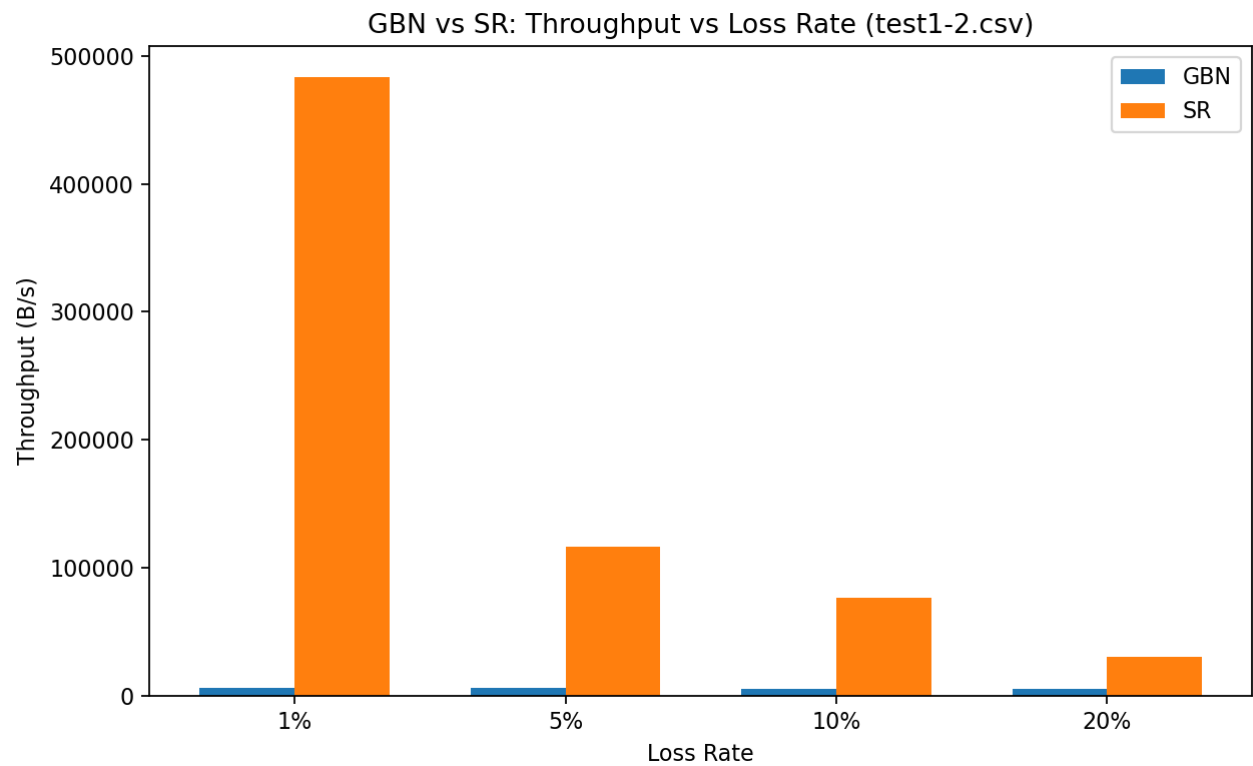
- `ssthresh` 初值影响慢启动阶段长度；可根据网络容量调优。
- `alpha/beta` 影响 Vegas 的敏感度；丢包率高时适当增大 `beta` 可减少过度收缩。
- `timeout_ms` 应与网络 RTT 量级匹配，过小会导致频繁重传，过大则恢复慢。

此次实验中，参数均取定值。

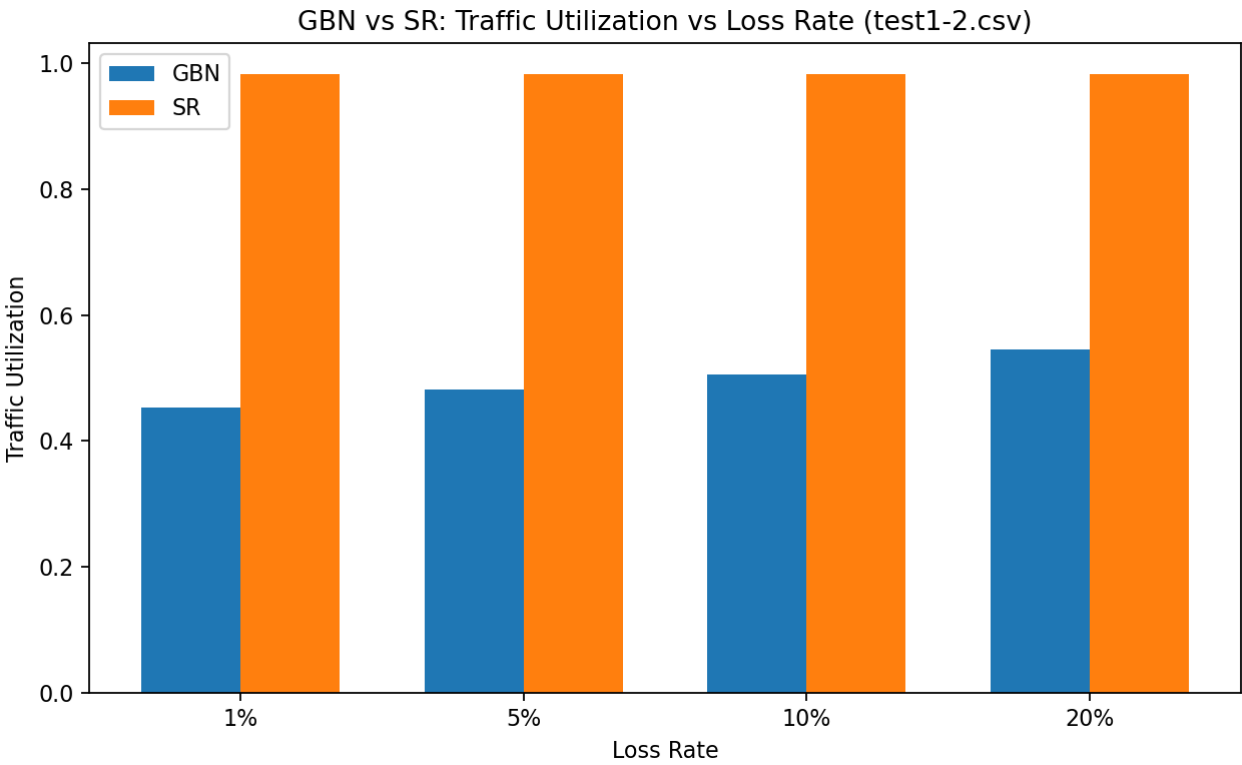
实验结果与分析

原始数据位于 `/test-data` 中的 `test1-2.csv`、`test3-4.csv`、`test5-6.csv`、`test7-8.csv`、`test9-10.csv`。

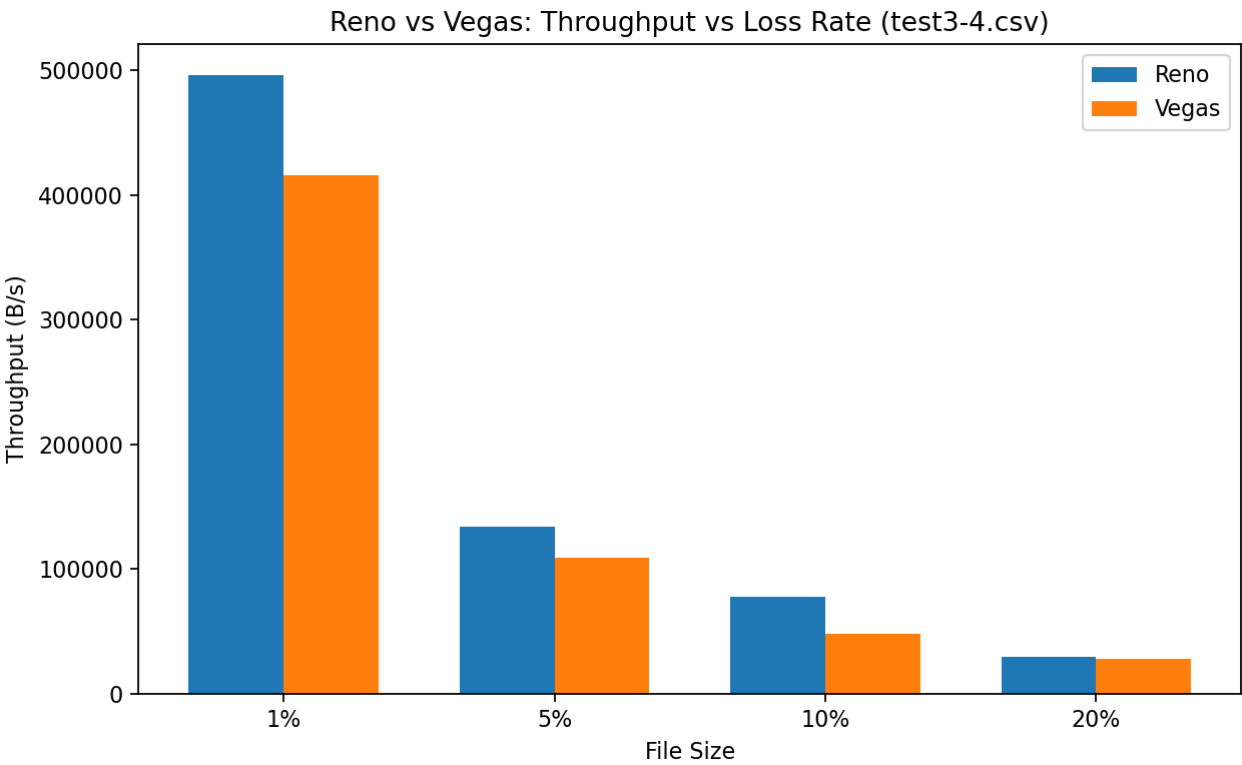
1. 规律：模拟环境下上传文件丢包率越高，有效吞吐量越低，基本成反比关系；SR 算法的有效吞吐量远大于 GBN 算法。



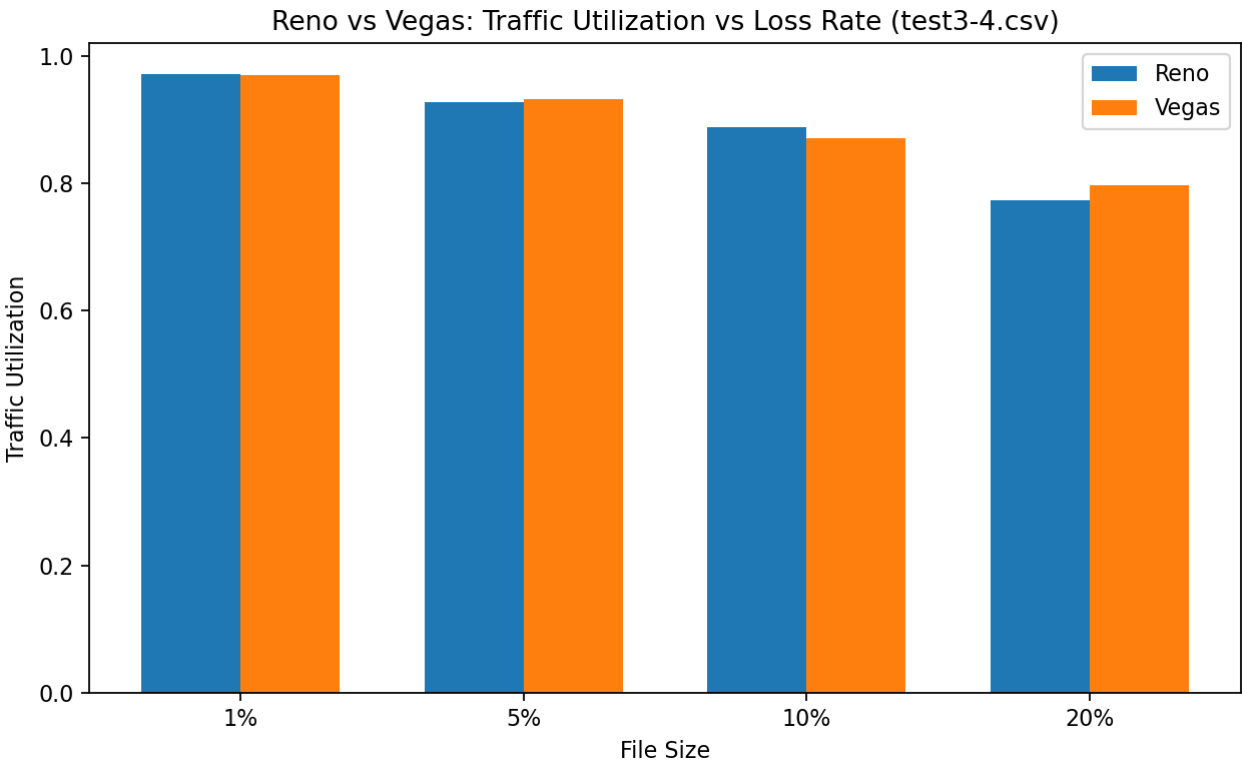
2. 规律：模拟环境下上传文件丢包率变化时，流量利用率基本不变；SR 算法的流量利用率远大于 GBN 算法，且维持在稳定值。



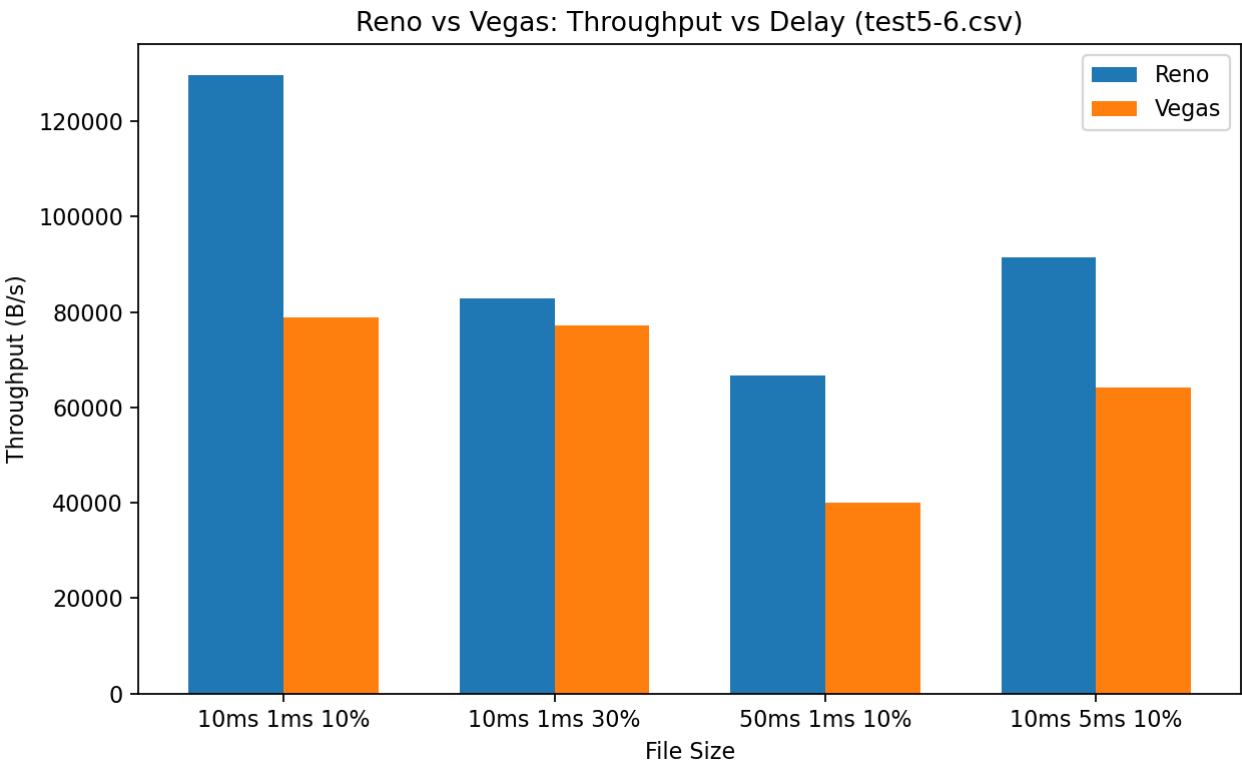
3. 规律：模拟环境下下载文件丢包率越高，有效吞吐量越低，基本成反比关系；Reno 算法的有效吞吐量大于 Vegas 算法。



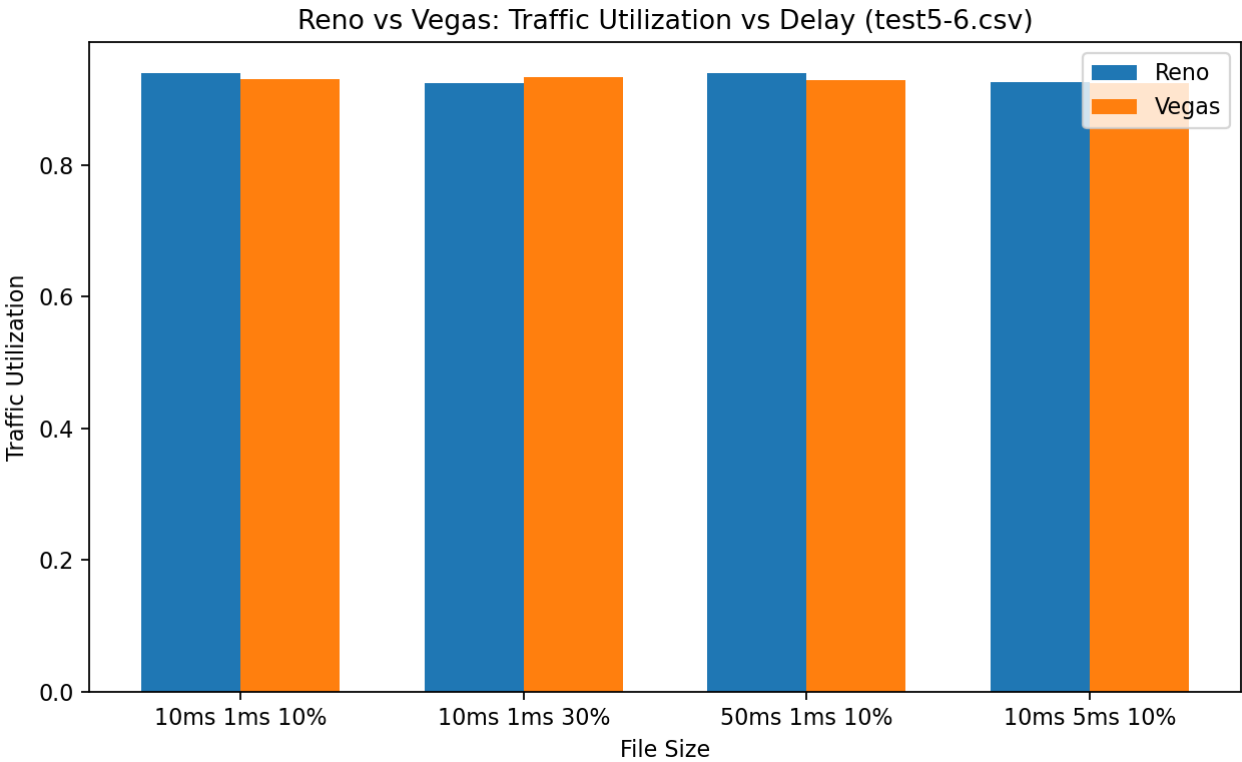
4. 规律：模拟环境下下载文件丢包率变化时，流量利用率下降；Reno 算法的流量利用率稍稍大于 Vegas 算法，且维持在稳定值。



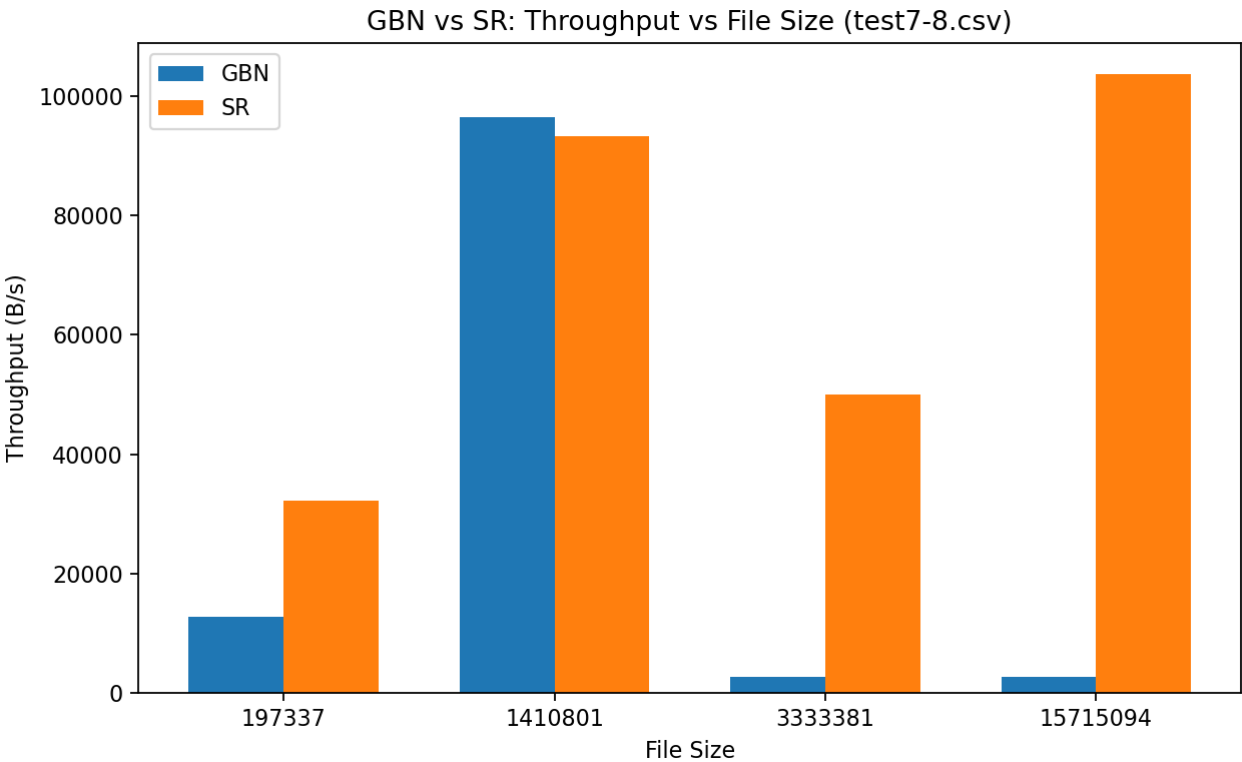
5. 规律：模拟环境下，延迟、抖动、前后包延迟相关系数的增加均会导致有效吞吐量下降；Reno 算法的有效吞吐率下降较 Vegas 算法明显，但是 Reno 算法在各条件下表现好于 Vegas 算法。



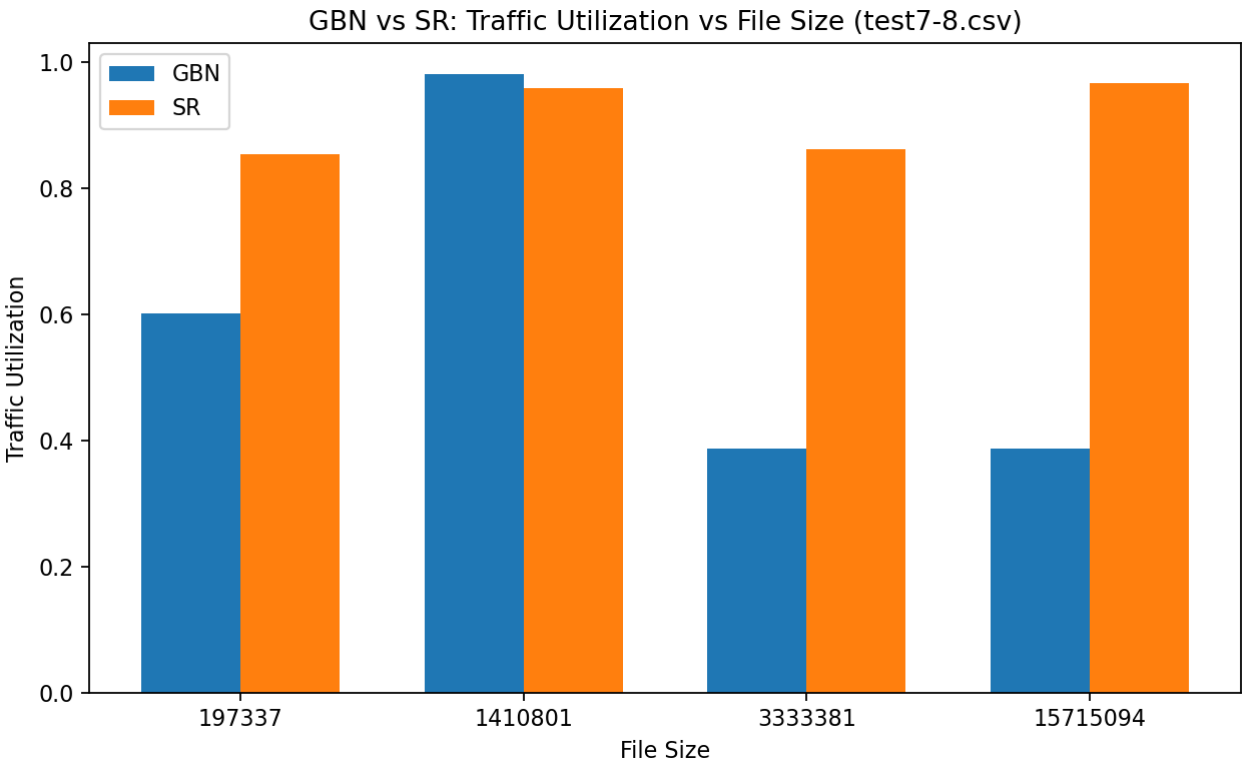
6. 规律：模拟环境下，延迟、抖动、前后包延迟相关系数的变化与流量利用率无关。



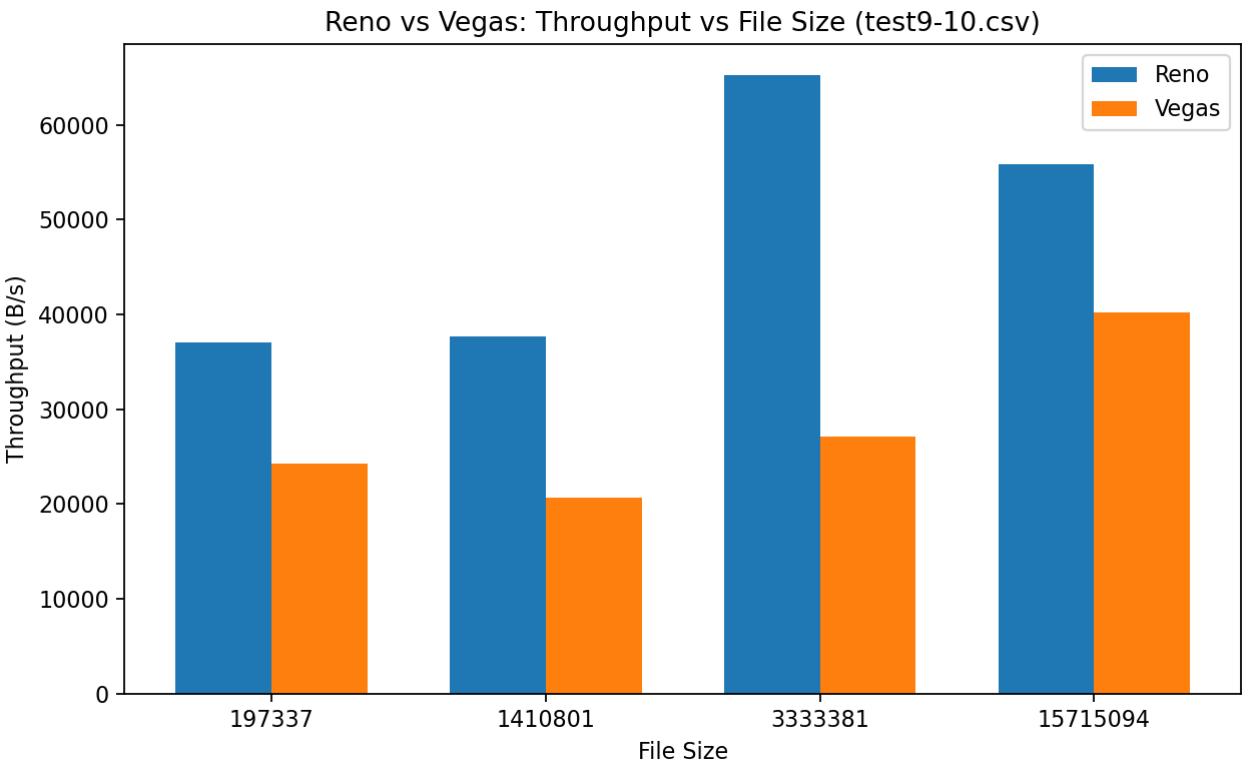
7. 规律：真实上传文件时，对 SR 算法来说文件大小越大，有效吞吐量越高，而对 GBN 算法来说反之。（对某个 1.4 MB 左右大小的文件存在例外情况）



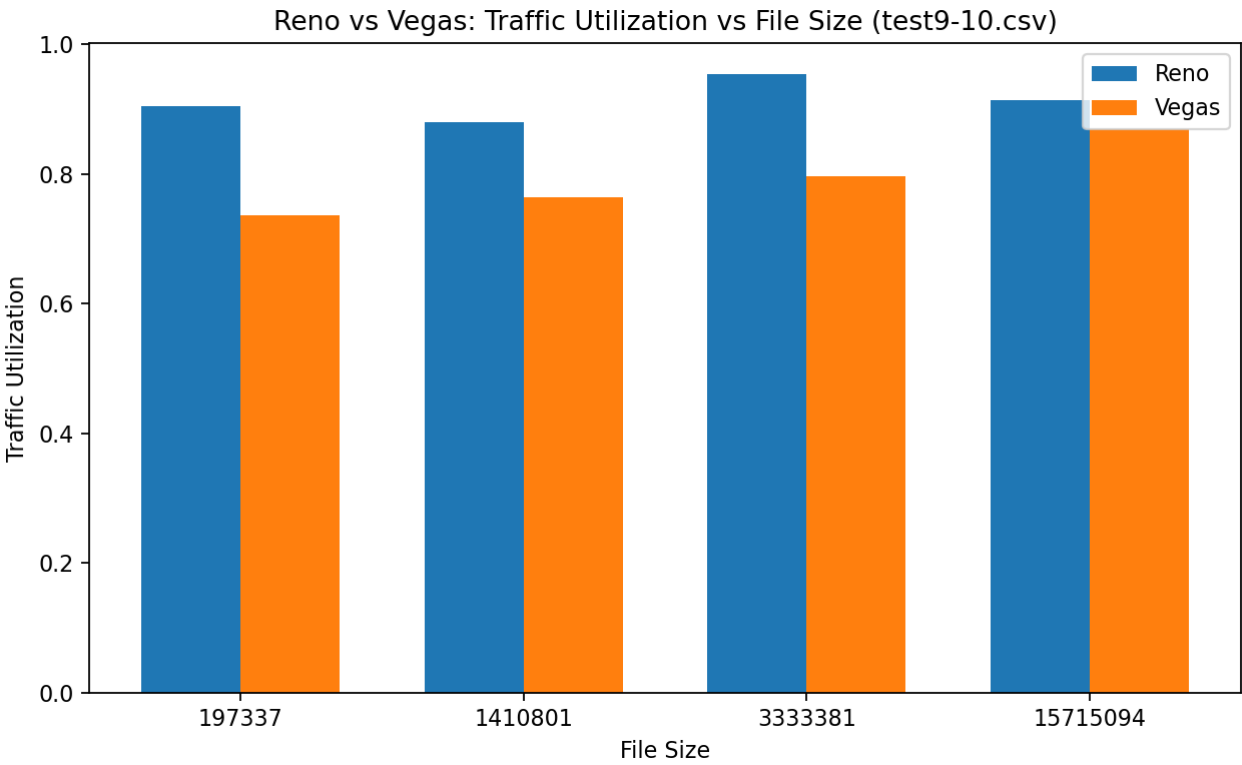
8. 规律：真实上传文件时，对 SR 算法来说流量利用率的变化与文件大小无关；对 GBN 算法来说，文件大小越大流量利用率越低。（依然对某个 1.4 MB 左右大小的文件存在例外情况）



9. 规律：真实下载文件时，Reno 算法有效吞吐量在文件大小变大时先变大后变小；Vegas 算法有效吞吐量在文件大小变大时先变小后变大；总体上 Reno 算法有效吞吐量大于 Vegas 算法。



10. 规律：真实下载文件时，Reno 算法流量利用率在文件大小变化时波动但是总体持平；Vegas 算法流量利用率在文件大小变大时略微变大；总体上 Reno 算法流量利用率稍大于 Vegas 算法。



总体规律：

- SR 重传策略和基于丢包的 Reno 拥塞控制算法组合显著优于其他组合（SR 策略优于 GBN 策略，Reno 算法优于 Vegas 算法）；
- 有效吞吐量由算法、网络环境和文件大小因素共同决定，与网络环境（尤其是丢包率）强相关；
- 流量利用率一般来说由算法决定，网络环境和文件大小导致其波动，没有决定性影响。